11 Datenbanken

1. Motivation
2. Datenorganisation und Datenbankkonzept
3. Semantische Datenmodellierung
4. Umsetzung in Datenbanken
5. Datenbanknutzung mit SQL
6. Transaktionsmanagement
7. Datenbankentwicklung
8. Datenbanken und IT-Sicherheit
9. Systemarchitektur
10. NoSQL Datenbanken
11. Verteilte Datenbanken

11 Lernziele

- Sie wissen, wie ein DBS in eine Applikationssoftware eingebunden wird und eine Datenbasis implementiert werden kann.
- Sie wissen, was der Begriff „Big Data“ bedeutet und welche Herausforderungen sich damit ergeben.
- Sie wissen, wie verteilte Datenbanken funktionieren und welche Vorteile diese haben.
- Sie kennen die Probleme, die sich beim Betrieb verteilter Datenbanken ergeben und wie sie sich lösen lassen.

11 Zugriffstechniken

- Zugriffstechniken der Programmiersprachen können variieren

Statement Level Interface (SLI)

- SQL Anweisungen eingebettet in Programmcode
- Verarbeitung durch Präprozessor
- Beispiele: eSQL/C, SQLJ

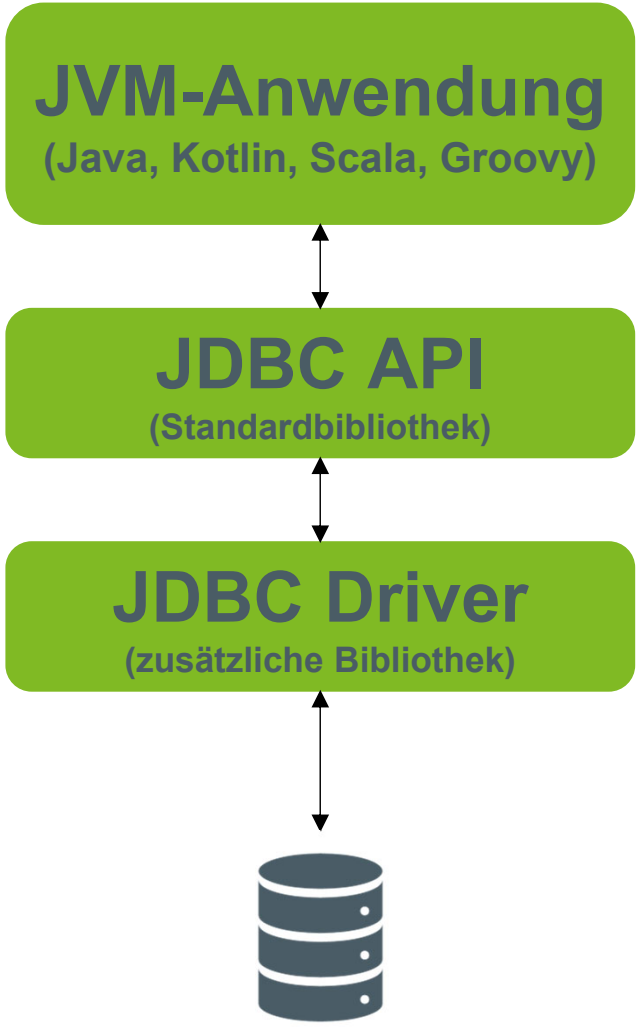
Call Level Interface (CLI)

- Bibliothek mit Zugriffsfunktionen
- Anwendungsprogramm stellt einen Datenbankbenutzer dar
- Beispiele: JDBC (JVM), ADO.NET (C#), ODBC (C, C++), NodeJS-Bibliotheken

11 Beispiel für CLI: JDBC

- Verwendung von SQL als String
- Gleichzeitiger Zugriff auf mehrere Datenbanken
- JDBC ist einfach zu lernen, schwer zu meistern
- „Adaptives“ Programmiermodell
- Grundlage für andere Techniken, wie:

  SQLJ, JPA, Exposed
- Ziele:
  - Einfachheit
  - Robustheit
  - Verfügbarkeit
  - Skalierbarkeit



Allgemeines Vorgehen mit JDBC




```
try {
// 1. Verbindung herstellen
Connection con = DriverManager
.getConnection(
    "jdbc:postgresql://localhost/rezepte",
    "benutzername", "passwort„
);
// 2. Statement erzeugen und ausführen
Statement stmt = con.createStatement();
// 3. Statement ausführen
ResultSet rs = stmt.executeQuery("select * from rezept");
// 4. Ergebnis verwenden
while (rs.next()) {
    System.out.println(rs.getString("titel"));
}
// 5. Ressourcen freigeben
rs.close(); stmt.close(); con.close();
} catch (Exception e) {
    e.printStackTrace();
}
```

Was macht das Programm?

11 Statements in JDBC



Statement

- Code wird als String an DBMS gesendet
- DBMS übersetzt, optimiert und führt aus

PreparedStatement

- Code wird mit Platzhaltern im String an DBMS gesendet
  1. DBMS übersetzt und optimiert
  2. Anweisung kann mehrfach mit wechselnden Parametern ausgeführt werden.

CallableStatement

- Aufruf von Stored Procedures

11 Beispiel für PreparedStatement



- Über eine Suchfunktion werden Rezepttitel eingegeben
- Platzhalter in SQL Statement werden an DBMS gesendet
- DBMS erstellt Zugriffsplan

```
PreparedStatement pstmt = conn.prepareStatement(
  "select * from rezept r where r.titel = ?;"
);
while (true) {
  String rezeptTitel = JOptionPane.showInputDialog(
                                          "Rezeptname eingeben:");
  pstmt.setString(1, rezeptTitel);
  ResultSet rs = pstmt.executeQuery();
  if (rs.next()) {
    System.out.printf("Rezept %s ist von %d und dauert %d min.\n",
       rezeptTitel, rs.getInt("benutzer_id"), rs.getInt("dauer_min"));
  } else {
    System.err.printf("Rezept %s wurde nicht gefunden!\n",
                                                  rezeptTitel);
  }
}
```

11 Vorteile des PreparedStatement



- Schnellere Ausführung gleichartiger Abfragen
- Sauberer Code
  - Statt String-Verkettung werden Variablen eingesetzt
  - Das Statement wird nur an einer Stelle „gebaut“
- Keine SQL-Injection möglich!

Adaptives Programmieren mit JDBC / Metadaten



- Über Interfaces, kann die Struktur der DB (Schema) abgefragt werden

ResultSetMetaData → Informationen über die Ergebnismenge

- int getColumnCount()
- String getColumnName(int column)
- int getColumnType(int column)

DatabaseMetaData → Informationen über das DBMS und das Schema

- String getDatabaseProductName()
- ResultSet getTables(…)
- ResultSet getColumns(…, String tableName)
- String getTimeDateFunctions()
- … und sehr viele mehr.

11 Datentypen von SQL → JDBC



SQL Typ	Java Typ
INT, INTEGER, SERIAL	int
VARCHAR	String
CHAR	String
TEXT	String
DECIMAL, NUMERIC	java.math.BigDecimal
FLOAT, DOUBLE	double
DATE	java.sql.Date
TIME	java.sql.Time
TIMESTAMP	java.sql.Timestamp
BLOB	byte[]

Datenmanipulation mit JDBC



Einfachstes Vorgehen: executeUpdate()

- UPDATE / INSERT wird als (Prepared)Statement mit executeUpdate() abgesetzt
- executeUpdate() liefert dann kein ResultSet, sondern die Anzahl der veränderten Zeilen

Mächtigeres Vorgehen: ResultSet verwenden

- Mit JDBC 2.0 verfügt ein ResultSet über eine updateXXX()-Methode für jeden Datentyp
- Beispiel:

```
rs.updateString(1, "mystring");
rs.updateFloat(2, 11000.0f);
```

11 JDBC – ResultSet – Beispiel



Lesen oder Schreiben:

```
statement = conn.createStatement(
  ResultSet.TYPE_SCROLL_INSENSITIVE, ResultSet.CONCUR_UPDATABLE);
rs = statement.executeQuery("select * from rezept;");
```

TYPE_FORWARD_ONLY

Kann nur vorwärts durchlaufen werden (Schnappschuss / „Live Update“)

TYPE_SCROLL_INSENSITIVE

Vorwärts und Rückwärts

Zeigt nur die Daten, die vorhanden waren beim Absenden des Statements

TYPE_SCROLL_SENSITIVE

Vorwärts und Rückwärts

Es werden Daten zwischendurch erfolgter Updates sichtbar

Navigation:

- previous() / next(): Springt zur vorherigen /nächsten Zeile
- First() / Last()
- beforeFirst() / afterLast()
- relative(int rows) / absolute(int r)

JDBC - ResultSet

Datenzugriff auf RS ist immer auf eine Zeile der Ergebnismenge.

Werte lesen

- `rs.getString(1)`
- `rs.getString("titel")`

Werte ändern

- `rs.updateBoolean("privat", true)`
- `rs.updateRow()`

Datensätze einfügen

- `rs.moveToInsertRow()`
- `rs.updateString("titel", "Cheeseburger mit Käse")`
- …
- `rs.insertRow()`

Transaktionen mit JDBC



- Transaktionsmodus liegt an der jeweiligen JDBC-Verbindung / Session
- Einfach: Auto-Commit-Modus
  - Jede einzelne SQL-Anweisung führt zu einem Commit
  - `conn.setAutoCommit(false);`
- Nun kann man das Transaktionsende selbst bestimmen!
  - `conn.commit();`
  - `conn.rollback();`

11 Isolationslevel in JDBC



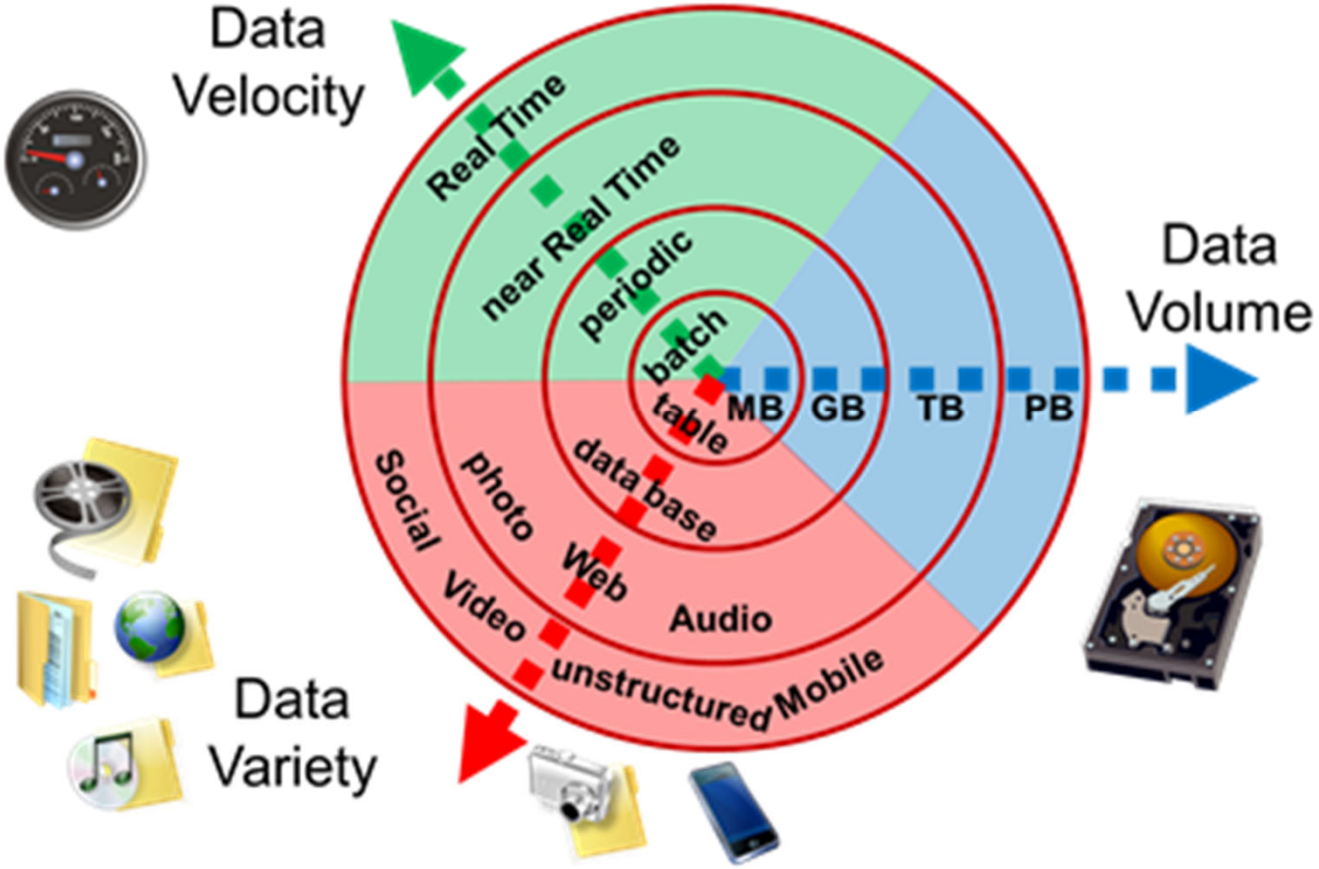
- TRANSACTION_NONE
- TRANSACTION_READ_UNCOMMITTED
- TRANSACTION_READ_COMMITTED (Default in JDBC)
- TRANSACTION_REPEATABLE_READ
- TRANSACTION_SERIALIZABLE

Beispiel:

```
DatabaseMetaData d = con.getMetaData();
if (d.supportsTransactionIsolationLevel(TRANSACTION_SERIALIZABLE) {
  Connection.setTransactionIsolation(TRANSACTION_SERIALIZABLE);
}
```

Aspekte von Big Data

- **Volume**: Große Menge der zu speichernden Daten
- **Velocity**: Hohe Geschwindigkeit der Erhebung neuer Daten
- **Variety**: Starke Variation in der Struktur der Daten



11 Globales Datenaufkommen & Future Skills



→**Im Jahre 2027:**
wird das Datenvolumen voraussichtlich auf über 284 Zettabytes ($285*11^{21}$ Bytes) ansteigen.

https://de.statista.com/statistik/daten/studie/267974/umfrage/prognose-zum-weltweit-generierten-datenvolumen/#professional
Abruf 01.11.23

McKinsey - Studie: zusätzlich benötigte Mitarbeiter bis 2023 im D.
Platz 1: Komplexe Datenanalyse: 455 Tausend
Platz 2: Nutzerzentriertes Designen (UX): 79 Tausend
…
https://www.stifterverband.org/future-skills/bedarf-bis-2023

Future Job Skills: Datenkompetenz bis 2030 gefragteste Fähigkeit
https://www.digital-chiefs.de/datenkompetenz-job-skill-bis-2030-faehigkeit/ **Meldung vom Juli 2022**

11 Big Data – Herausforderungen

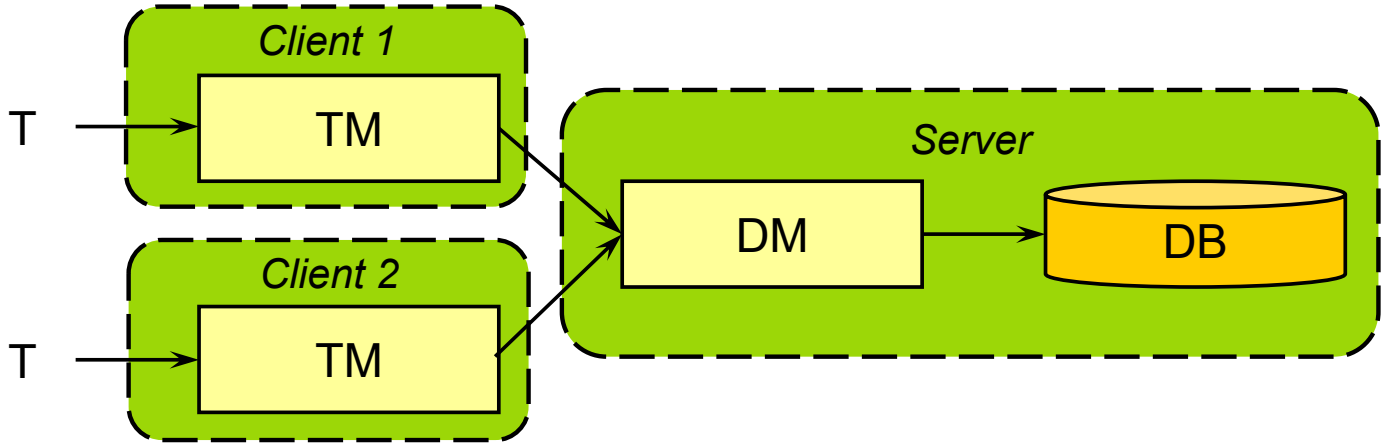
- Große Datenmengen stellen Unternehmen vor neue Herausforderungen im Umgang mit Big Data
- Mobile Endgeräte
  - Zahl mobiler Endgeräte wächst zweistellig
  - Viele Produkte im Bereich der personalisierten Medizin (Smartwatches, Wearables)
- Internet of Things (IoT)
  - Generierung massiver Datenmengen
- Alle diese Anwendungen benötigen hochperformante, verteilte und robuste Datenbanksysteme

Big Data – Skalierung der Hardware

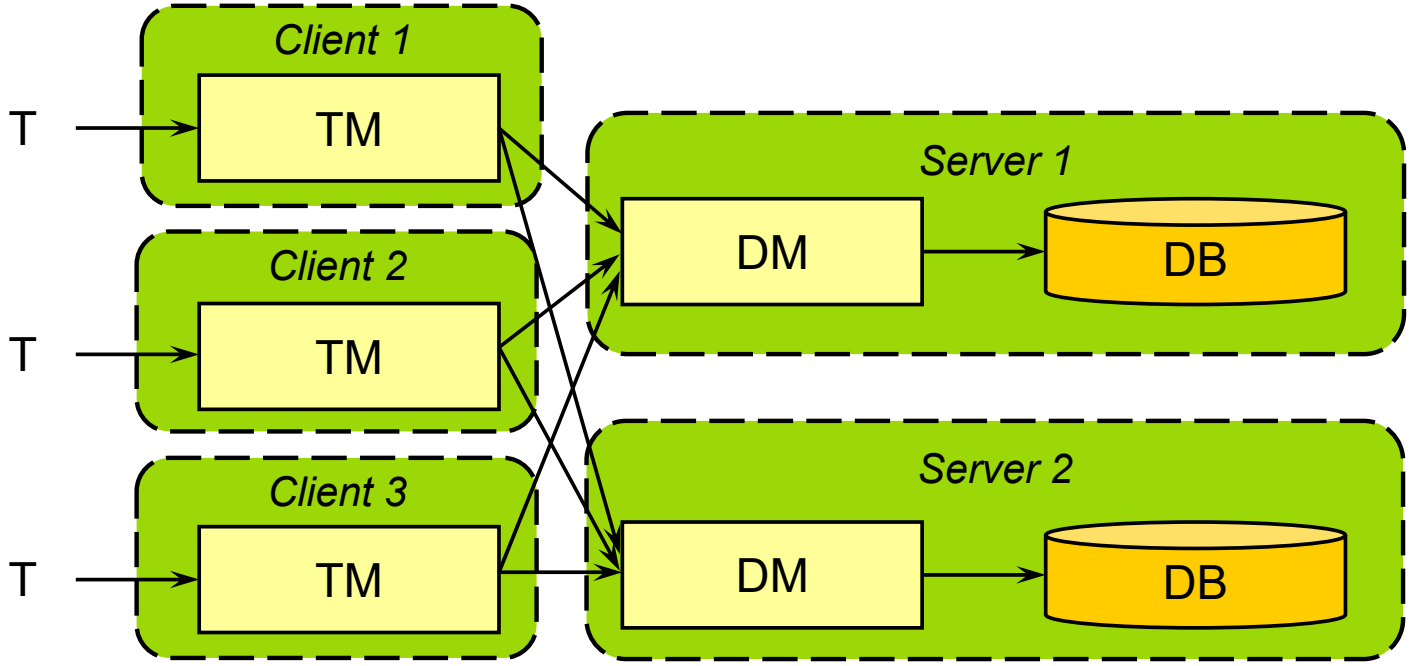
- **Scaling Up (Vertikale Skalierung)**
  - Alle Server erfahren das gleiche Upgrade (schnellere CPU, mehr RAM,...)
  - Mehr Ressourcen pro Server
- **Scaling Out (Horizontale Skalierung)**
  - Verteilung der Last auf mehrere Server (Clustering)
  - 1100 x 1TB Festplatten sind kostengünstiger als eine PB Festplatte
  - Mehr Server im System



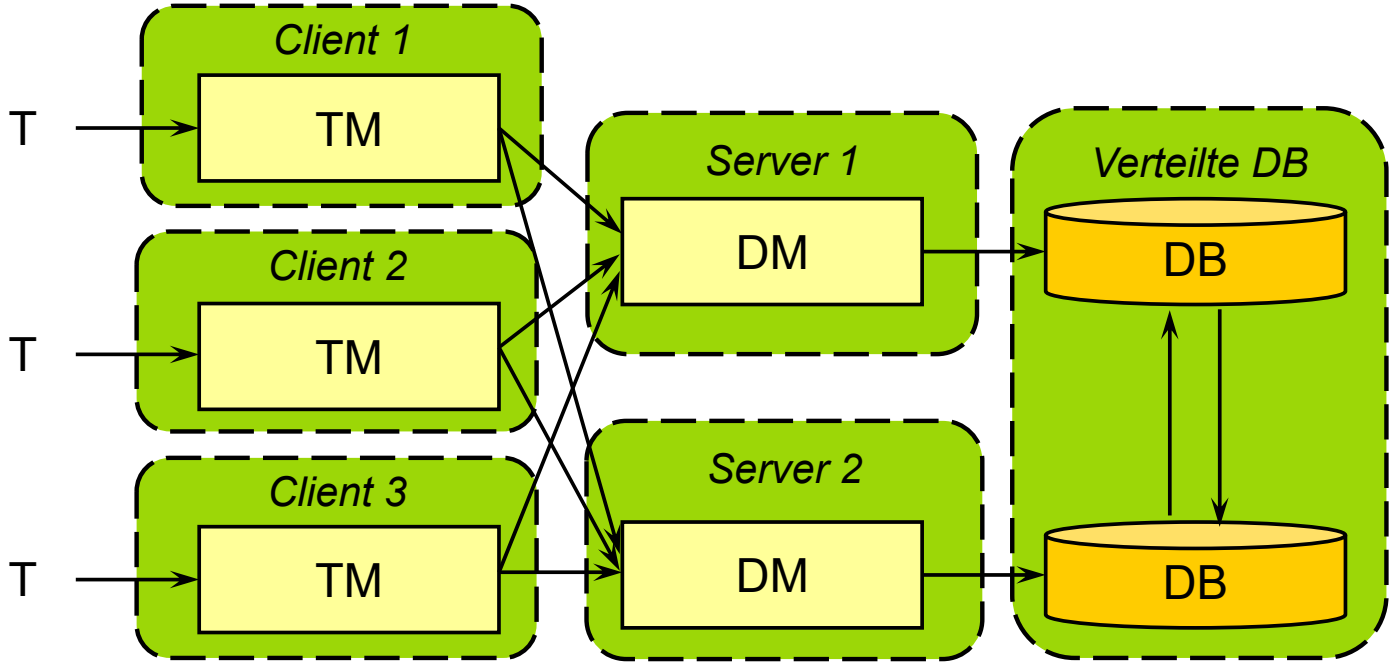
Single-Site Data (Client-Server-Architektur)



Multiple-Site Data (Verteilte Architektur)



Multiple-Site Data (Verteilte Architektur)



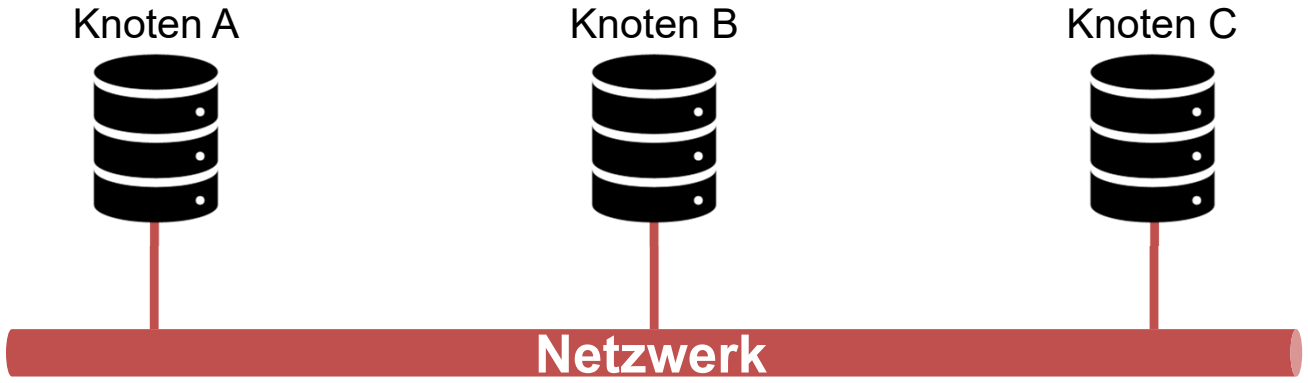
11 Verteiltes Datenbanksystem

Ein **verteiltes Datenbanksystem** besteht aus mehreren über ein Datennetz verbundenen Knoten, für die gilt:

- Jeder Knoten stellt eine eigenständige Datenbank dar, aber
- die Knoten arbeiten bei Bedarf zusammen, so dass jeder Benutzer an jedem Knoten auf jedes Datum, das im Netzwerk existiert, so Zugriff hat, als ob das Datum am Ort des Benutzers gespeichert wäre.

11 Eigenschaften verteilter Datenbanken

- Eine verteilte Datenbank besteht aus mehreren Knoten.
- Diese Knoten…
  - Sind über ein Netzwerk verbunden
  - Agieren als eigenständige Datenbanken
  - Arbeiten zusammen, um Anfragen an die **verteilte Datenbasis** zu erfüllen
- Der Benutzer merkt nicht, dass die Datenbank verteilt ist. (Transparenz)
  - Es ist kein Wissen des Anwenders über die Verteilung erforderlich



11 Verteilte Datenbasis

Eine **verteilte Datenbasis** ist eine Sammlung von Daten,

- die physisch über mehrere, autonom agierende, über ein Netzwerk verbundene Rechner verteilt sind;
- die durch die überwiegende Zahl von Anwendungen rechnerlokal bearbeitet werden;
- deren Zusammenschluss eine einheitliche, logische Gesamtsicht auf die Daten darstellt;
- die so verwaltet werden, dass jedes beliebige Anwendungsprogramm den Eindruck hat, die Daten würden auf dem Rechner gehalten, auf dem das jeweilige Anwendungsprogramm abläuft.
- auf die in ihrer Gesamtheit von jedem Rechner des Netzwerks aus zugegriffen werden kann.

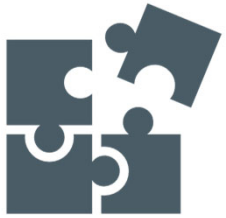
Weitere Infos: Mehrrechner-Datenbanksysteme - Grundlagen der verteilten und parallelen Datenbankverarbeitung, Erhard Rahm, 1994
https://www.researchgate.net/publication/220688530

11 Motivation für verteilte Datenbanken



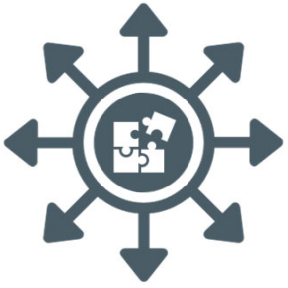
Replication / Replikation

Duplizierung von Daten über mehrere Knoten der verteilten Datenbank.
Funktion: Backup, horizontale Skalierung, Redundanz bei Serverausfällen



Fragmentierung

Aufteilung von großen Tabellen anhand von Attributen oder Spalten.
Funktion: Steigerung der Zugriffsgeschwindigkeit, „Archivierung“



Sharding / Aufteilung / Allokation

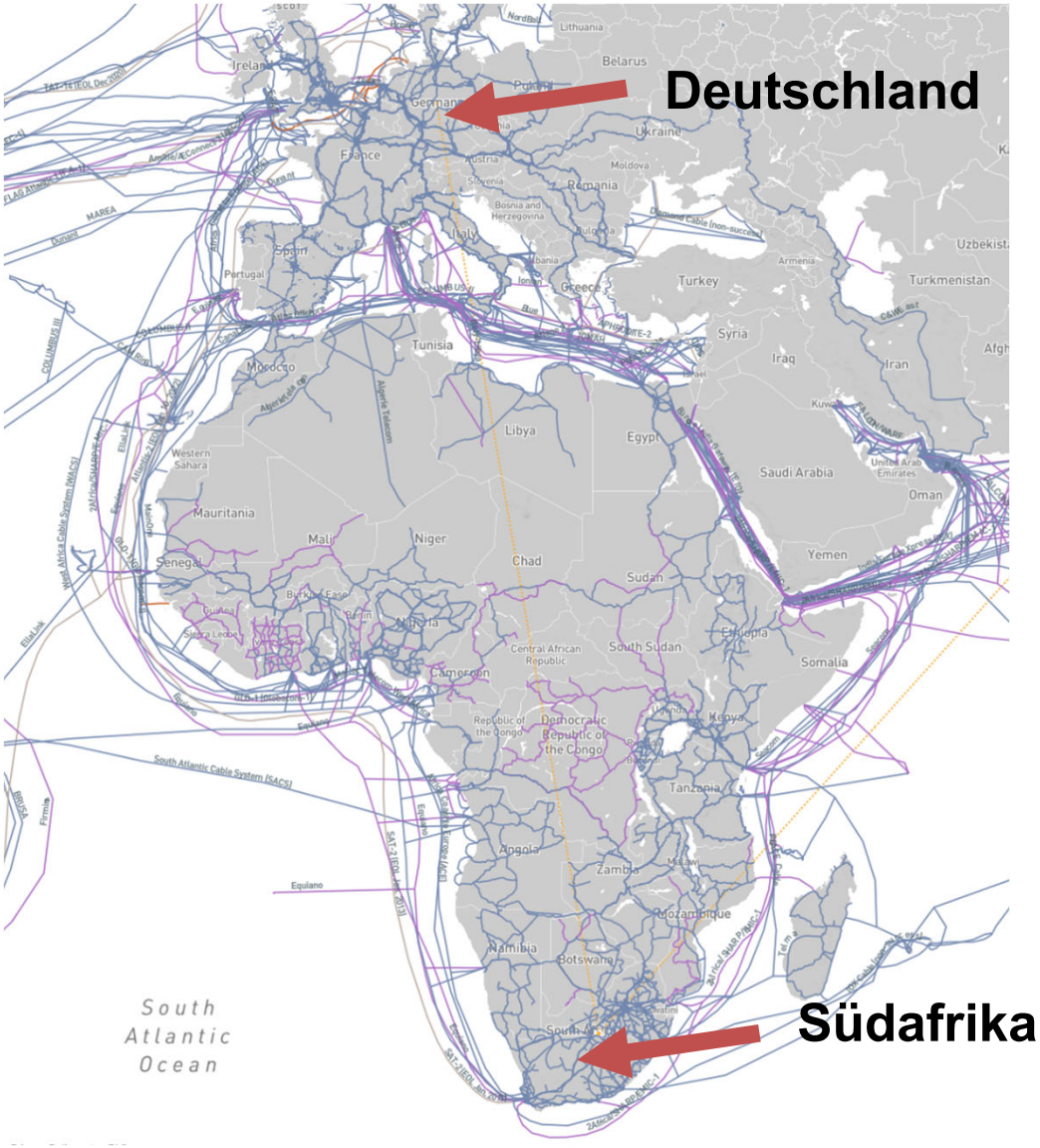
Aufteilung der ggf. vorher fragmentierten Datenbasis über mehrere Knoten.
Funktion: Datenlokalität bei weniger Redundanz, Lastverringerung

11 Geografische Verteilung (Extrembeispiel)

Daten …

- brauchen Zeit (Latenz, Bandbreite)
- gehen verloren (Paketverlust)
- gehen durch viele Knoten („Hops“)
- nutzen „langsame“ Protokolle (TCP)

➔ Datenlokalität ist sinnvoll



Karte: https://www.infrapedia.com/

11 Pfadverfolgung nach Südafrika

```
→ tracert www.gov.za

Routenverfolgung zu www.gov.za [164.151.129.20]
über maximal 30 Hops:

  1    <1 ms    <1 ms    <1 ms  WILHELM [192.168.169.1]
  2    23 ms    19 ms    12 ms  i5E86C64C.versanet.de [94.134.198.76]
  3    21 ms     7 ms     7 ms  i59F44416.versanet.de [89.244.68.22]
  4     7 ms     7 ms     7 ms  62.214.32.40
  5     8 ms     8 ms     8 ms  ae-22.r00.frnkge13.de.bb.gin.ntt.net [80.81.192.46]
  6    10 ms     7 ms     8 ms  ae-2.r20.frnkge13.de.bb.gin.ntt.net [129.250.6.13]
  7    18 ms    18 ms    18 ms  ae-14.r21.londen12.uk.bb.gin.ntt.net [129.250.3.12]
  8    20 ms    20 ms    33 ms  ae-1.a02.londen12.uk.bb.gin.ntt.net [129.250.3.215]
  9    22 ms    25 ms    21 ms  dimensiondata-0.r02.londen03.uk.bb.gin.ntt.net [83.231.235.222]
 10   186 ms   183 ms   187 ms  core1b-pkl-ten-ge-0-6-0-1.ip.ddii.network [168.209.100.213]
 11   183 ms   183 ms   183 ms  za-gp-pkl-hpe-1-fo-0-1-0-1-0sub102.ip.ddii.network [168.209.1.200]
 12   190 ms   190 ms   190 ms  za-gp-pkl-p-1-be-111.ip.ddii.network [168.209.129.136]
 13   183 ms   183 ms   183 ms  za-gp-bry-p-1-hu-0-0-0-11.ip.ddii.network [168.209.90.7]
 14   183 ms   183 ms   183 ms  za-gp-bry-pe-2-be-102.ip.ddii.network [168.209.131.135]
 15   188 ms   188 ms   188 ms  197.97.72.54
 16     *        *        *     Zeitüberschreitung der Anforderung.
 17     *        *     183 ms  dmz1.gcis.gov.za [164.151.129.20]
```

Websiteaufruf von Deutschland: 1MB Daten – 16 Sekunden

11 Anforderungen an eine verteilte Datenbank

Übersicht:

- Generell: Logische Transparenz aus Benutzersicht
  („Fundamentalprinzip“)
- Spezielle Regeln für verteilte Datenbanken:
  - Lokale Autonomie
  - Unabhängigkeit lokaler Komponenten
  - Dauerbetrieb / hohe Verfügbarkeit
  - Lokale Transparenz
  - Fragmentierungstransparenz
  - Replizierungstransparenz
  - Verteilte Bearbeitung von Datenbankoperationen
  - Verteiltes Transaktionsmanagement
  - Hardware-, Betriebssystem-, Netzwerk- und DBMS-Transparenz

Quelle: Hermann Kudlich: Verteilte Datenbanken, 1992

11 Anforderungen an eine verteilte Datenbank

- Lokale Autonomie:
  - Jeder Standort kontrolliert und verwaltet autonom die am Standort gespeicherten Daten.
  - Kein Standort A ist in dem Maße von einem anderen Standort B abhängig, dass A gar nicht funktionieren würde, wenn ein Zugriff auf B nicht möglich ist.
- Unabhängigkeit lokaler Komponenten
  - Alle Standorte sind gleichberechtigt.
  - Keine zentrale Instanz.
- Dauerbetrieb / hohe Verfügbarkeit
  - Für Updateprozesse oder Hinzufügen neuer Standorte sollte der Betrieb an den vorhandenen Standorten nicht eingeschränkt werden.
  - Unanfälligkeit gegenüber Fehlern wie z.B. Ausfall eines Knotens.

11 Anforderungen an eine verteilte Datenbank

- Lokale Transparenz
  - Ein Benutzer muss nicht wissen, an welchem Standort die Daten gespeichert sind.
  - Die Daten müssen so erscheinen, als seien sie alle am lokalen Ort gespeichert.
- Fragmentierungstransparenz
  - Für den Benutzer einer verteilten DB soll die Fragmentierung unsichtbar (transparent) sein.
    Beispiel: Eine Tabelle „Kunde“ ist fragmentiert.
    Die Abfrage kann trotzdem so formuliert sein:
- ```
  select * from Kunde
  where name = 'Müller';
  ```
- Replizierungstransparenz
  - Für den Benutzer soll die Replikation unsichtbar sein.
  - Die Daten auf allen Replikationen müssen gleich sein.
  - Die Replikation wird „automatisch“ gewählt.
  ```

11 Anforderungen an eine verteilte Datenbank

- Verteilte Bearbeitung von Datenbankoperationen
  - Benötigte Daten können an unterschiedlichen Orten liegen.
  - Es sind Techniken für die Optimierung der Datenabfragen notwendig, um die Netzlast zu verringern (sog. Query Optimizer).
- Hardwaretransparenz
  - Das verteilte Datenbankmanagementsystem (VDBMS) soll unabhängig von der zugrundeliegenden Hardware arbeiten.
  - Das VDBMS soll sich auf jeder Hardware gleich verhalten.
- Betriebssystemtransparenz
  - Das VDBMS soll unabhängig vom verwendeten Betriebssystem sein und sich auf verschiedenen Systemen gleich verhalten.
- Netzwerktransparenz
  - Das VDBMS soll unterschiedliche Netzwerktypen unterstützen.
- DBMS-Transparenz
  - Unterschiedliche SQL-Datenbankimplementierungen sollen in einem Netzwerk aufgrund des SQL-Standards zusammenarbeiten können.

11 Vorteile verteilter Datenbanken

- Datenlokalität
  - Die Daten liegen an dem Ort, an dem sie am meisten benötigt werden
- Zugriffsgeschwindigkeit
  - Schnellerer Zugriff auf die Daten, da meist nur eine Teilmenge der Gesamtdaten benötigt werden
- Bearbeitungseffizienz
  - Effizientere Datenbearbeitung, da dezentral
- Ausbaukosten
  - Hinzufügen neuer Knoten ist günstiger als Ausbau des eines Servers
- Ausfallsicherheit
  - Durch Verteilung der Daten höhere Ausfallsicherheit
- Erweiterbarkeit
  - Neue Standorte können in das Netzwerk integriert werden, ohne dass andere Standorte davon beeinträchtigt werden

11 Historie der verteilten Datenbanken

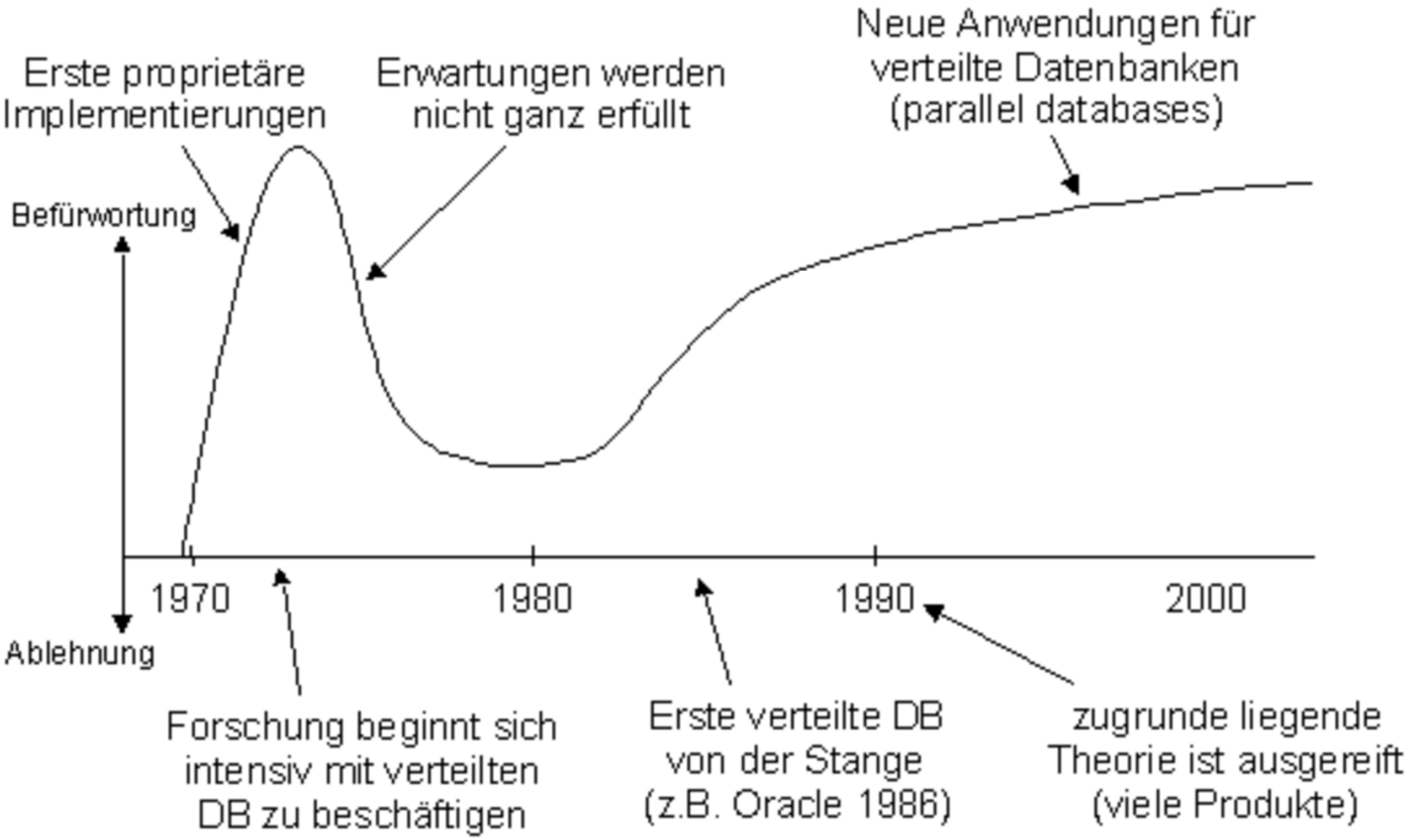


Bild: https://igw.tuwien.ac.at/fit/fit08/team3/distributed.html

11 Nachteile verteilter Datenbanken

- Management und Kontrolle
  - Weitaus komplexer als bei einer zentralen Datenbasis
- Sicherheit
  - Mehr „Angriffsfläche“ führt zu erhöhten Sicherheitsmaßnahmen
- Unterstützung heterogener Systeme
  - Mangelnde Einhaltung von Standards und/oder unterschiedliche Versionen erschweren den Zusammenschluss vorhandener DBMS zu einem verteilten DBMS
- Alle Probleme eines verteilten Systems

Vorgehen in **drei Phasen**

1. Entwurf des globalen Modells
2. Entwurf der Fragmentierung
3. Replizieren und Ortzuweisung

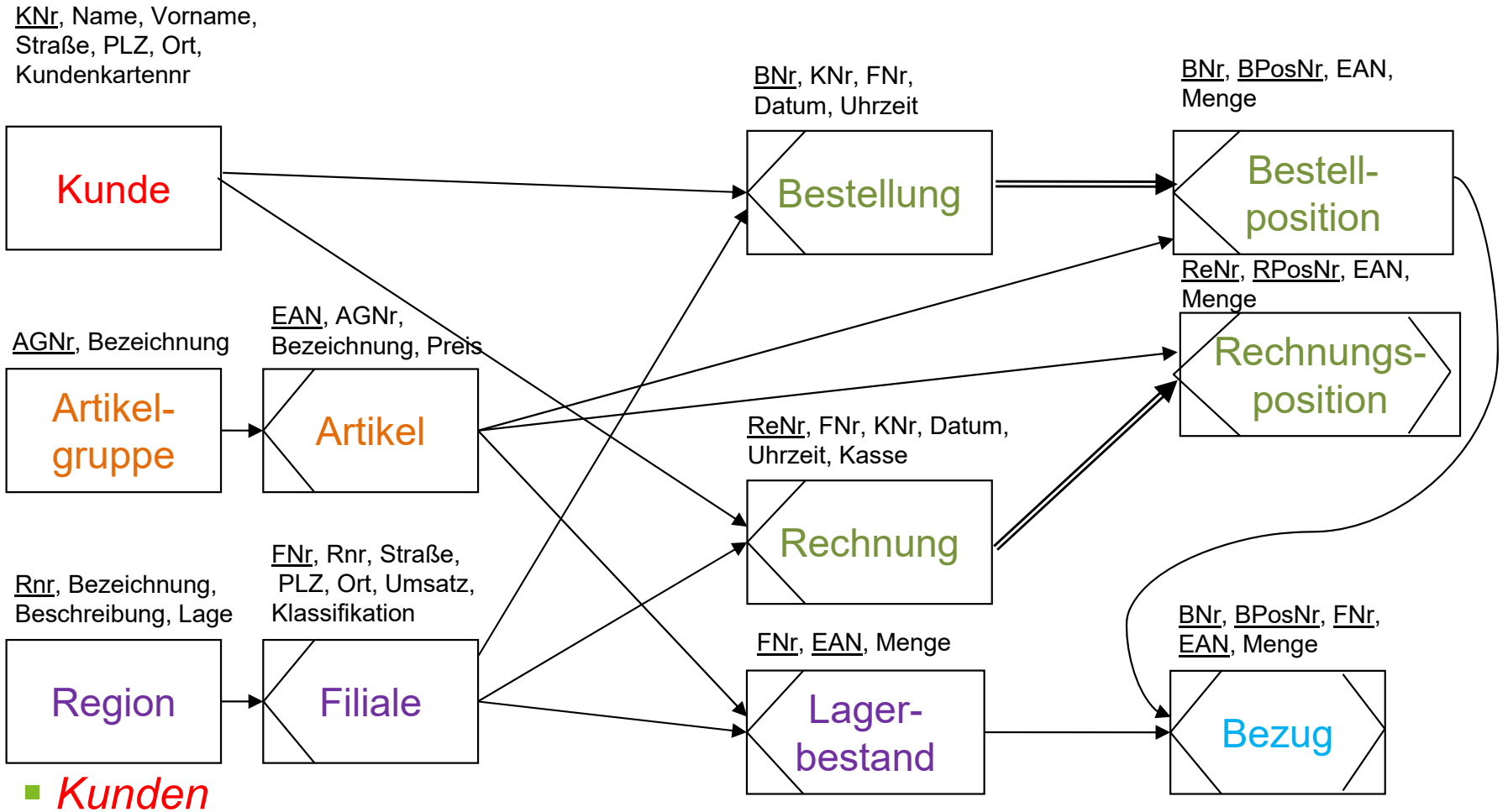
11 Beispiel

*Ein Unternehmen verfügt über mehrere **Filialen in verschiedenen Regionen**, deren aktueller **Artikelbestand** registriert wird. **Artikel sind in Gruppen zusammengefasst.** Verwenden Kunden beim Einkauf eine **Kundenkarte**, wird erfasst, wann sie **in welcher Filiale welchen Artikel** in welcher Menge bezogen haben. Darüber hinaus können auch **Artikel** bestellt werden, die nicht mehr in der Filiale vorhanden, jedoch noch **in anderen Filialen** in ausreichender Menge vorrätig sind. Nach der **Bestellung mit mehreren Positionen** erfolgt eine **Rechnung mit Positionen**.*

Zusammenfassung:

- *Kunden*
- *Regionale Filialen mit Artikelbestand*
- *Artikel sind gruppiert*
- *Bestellung/Rechnung hat Positionen*
- *Artikel können aus anderen Filialen bestellt werden*

11 Beispiel: Entwurf eines globalen Modells



- *Kunden*
- *Regionale Filialen mit Artikelbestand*
- *Artikel sind gruppiert*
- *Bestellung/Rechnung hat Positionen*
- *Artikel können aus anderen Filialen bestellt werden*

Wie sollen Daten jetzt fragmentiert werden?

11 Beispiel

Kunde									
KNr	Name	Vorname	Geschlecht	Strasse	PLZ	Ort	HNr	Kundengruppe	Umsatz

Region			
RNr	Bezeichnung	Beschreibung	Lage

Artikelgruppe	
AGNr	Bezeichnung

Filiale							
FNr	RNr	Strasse	HNr	PLZ	Ort	Umsatz	Klassifikation

Artikel			
EAN	AGNr	Bezeichnung	Preis

Bestand		
FNr	EAN	Menge

Rechnung					
ReNr	FNr	KNr	Datum	Uhrzeit	Kasse

11 Beispiel

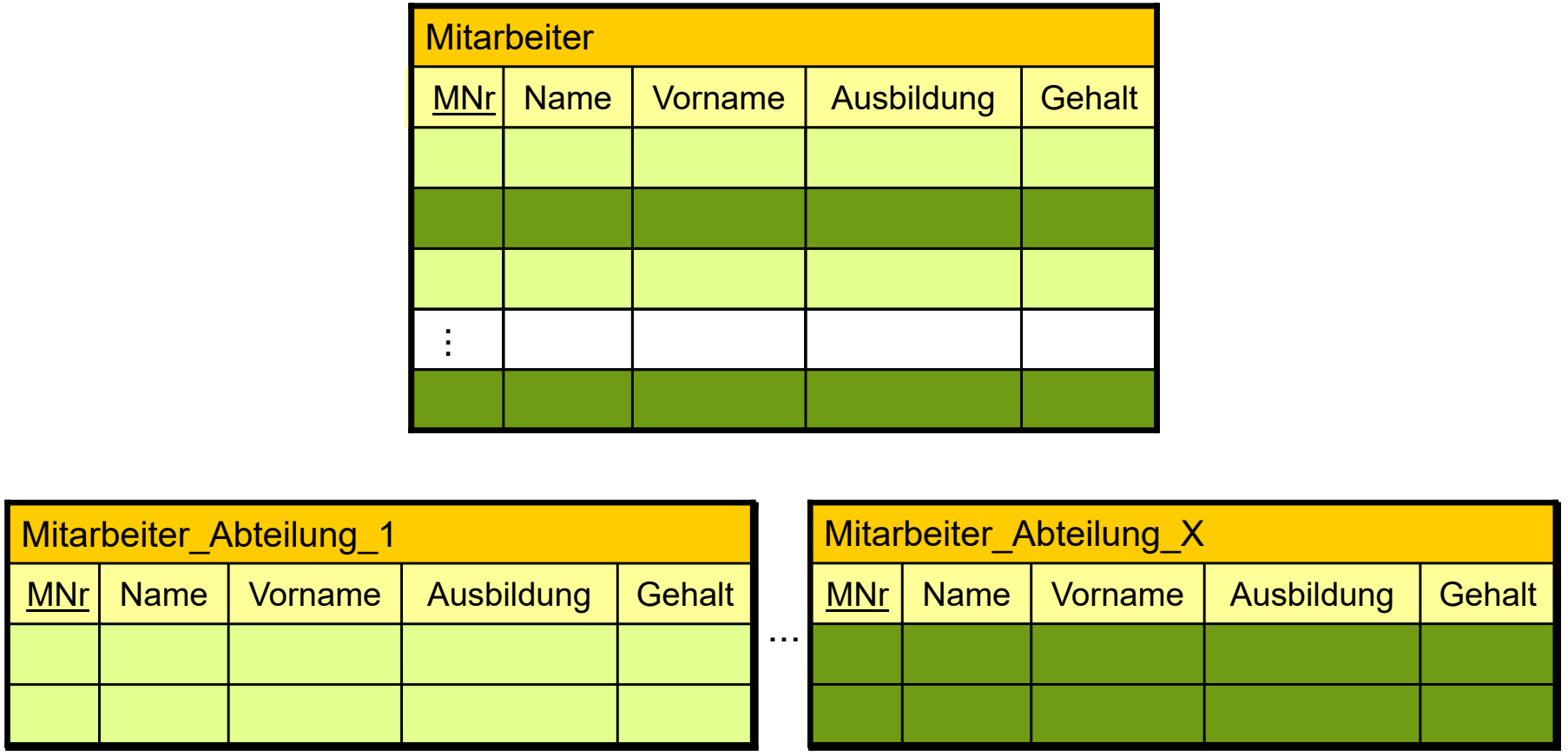
Rechnungsposition			
ReNr	RPosNr	EAN	Menge

Bestellung				
BNr	KNr	FNr	Datum	Uhrzeit

Bezug				
BNr	BPosNr	FNr	EAN	Menge

Bestellposition			
BNr	BPosNr	EAN	Menge

11 Beispiel: Horizontale Fragmentierung



11 Beispiel: Vertikale Fragmentierung

Kunde									
KNr	Name	Vorname	PLZ	Ort	Straße	HNr	Geb.Dat.	Geschl.	Familienstand

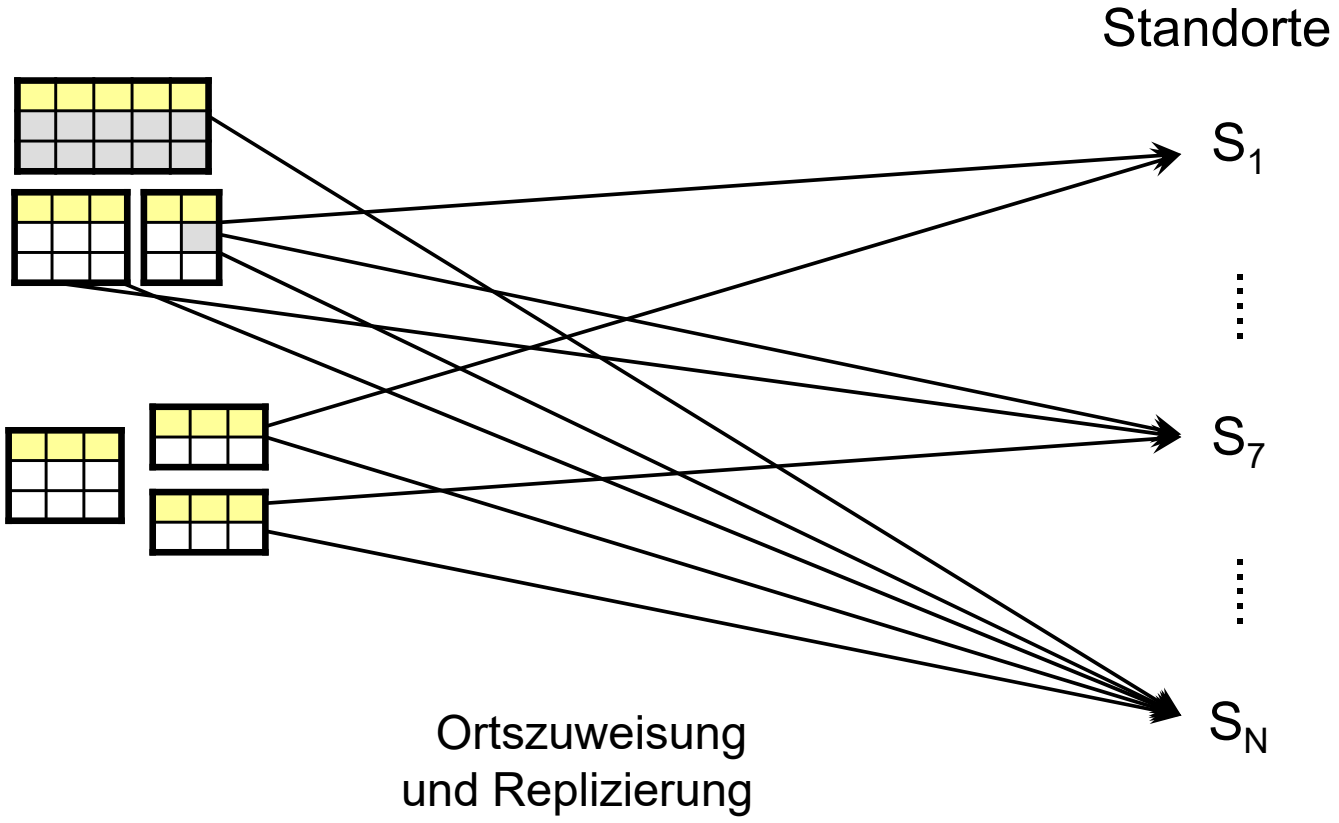
Kunde_Versand						
KNr	Name	Vorname	PLZ	Ort	Straße	HNr

Kunde_Marketing			
KNr	Geb.Dat.	Geschl.	Familienstand

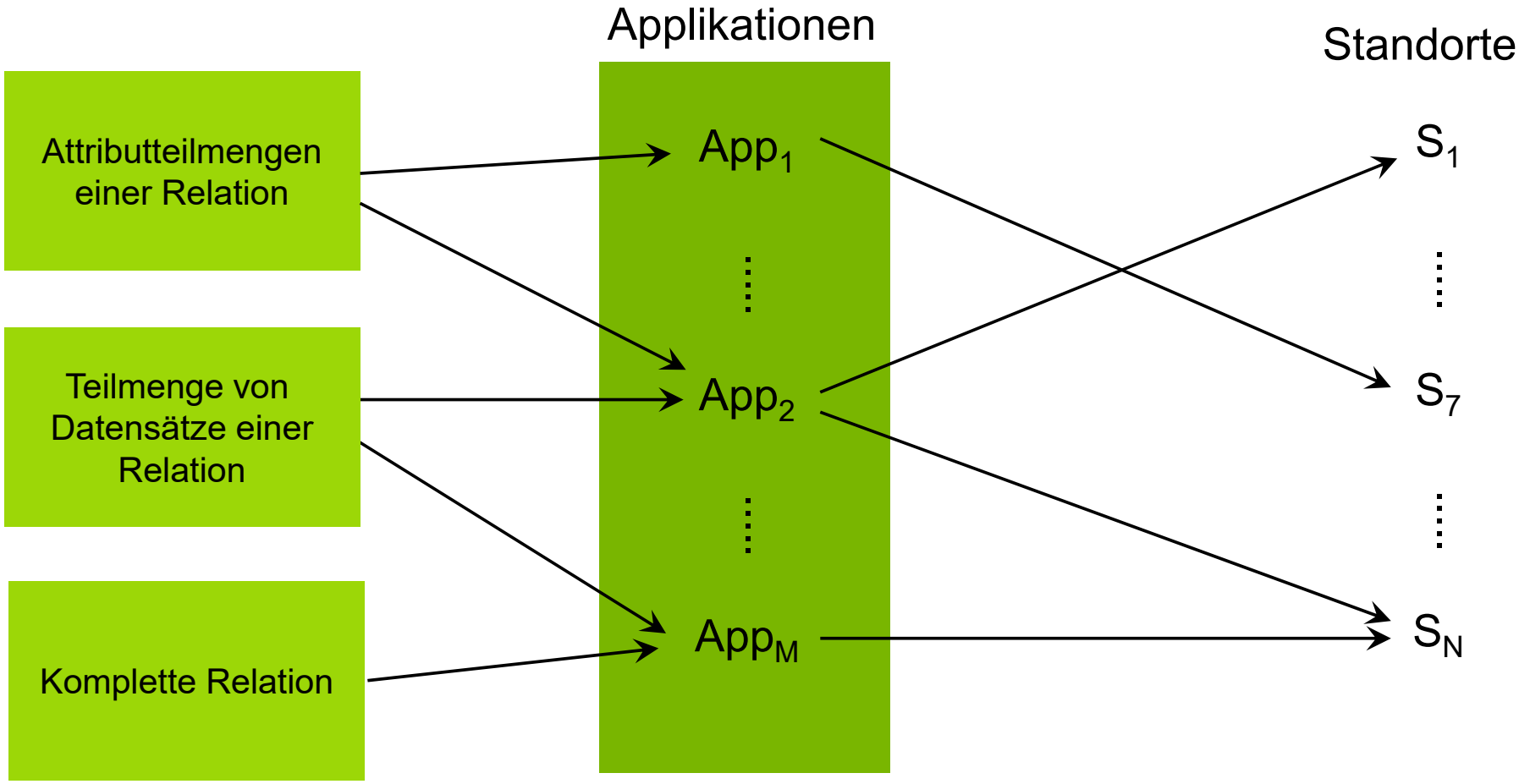
Fragmentierung, Ortzuweisung und Replizierung

Vertikale Fragmentierung
Attributteilmengen einer Relation

Horizontale Fragmentierung
Datensätze einer Relation



11 Beispiel: Replizierung und Ortzuweisung



11 Betrieb verteilter Datenbanken

Betriebsaufgaben:

1. Finden der relevanten Daten:
   Katalogmanagement
2. Sicherstellung der Konsistenz:
   Verteiltes Transaktionsmanagement



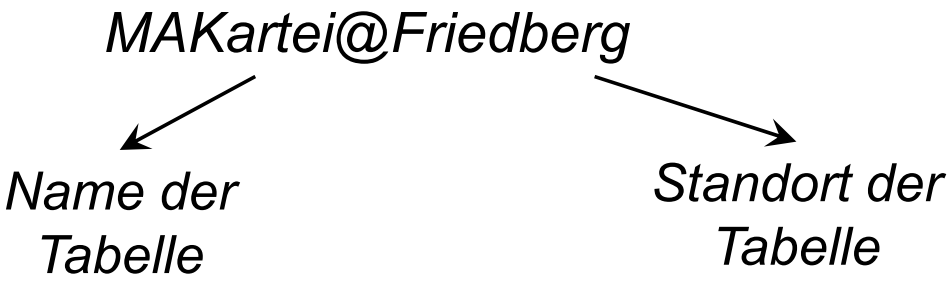
11 Katalogmanagement (Katalog = Schema)

Aufgabe: Finden des Standortes von Relationen

Lösungsmöglichkeiten:

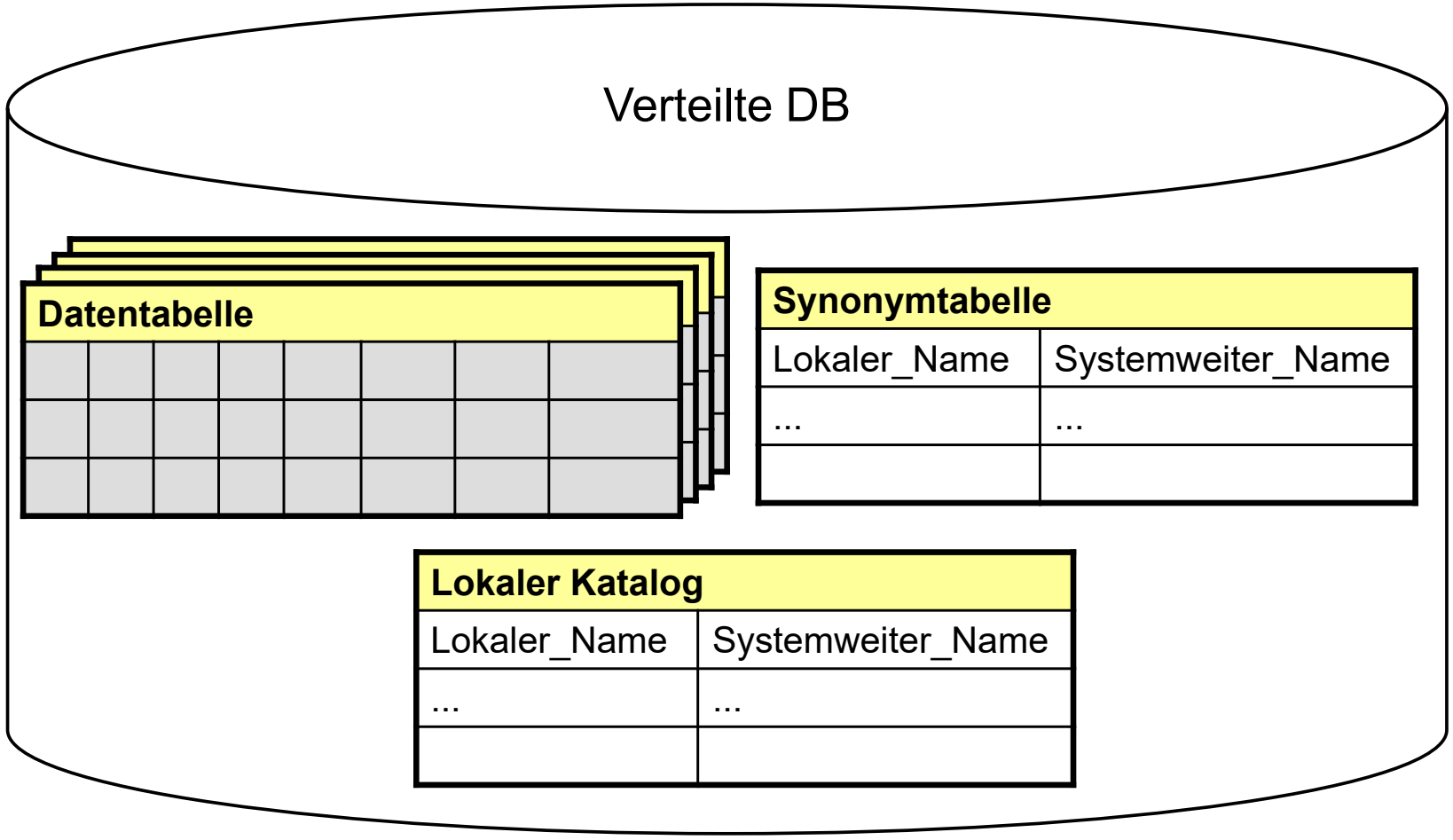
1. Zentralspeicherung des globalen Katalogs
2. Komplettkatalog an jedem Knoten
3. Kompromiss: Cluster aus mehreren Knoten haben globalen Katalog
4. Verteilte Katalog- und Katalogverweisverwaltung

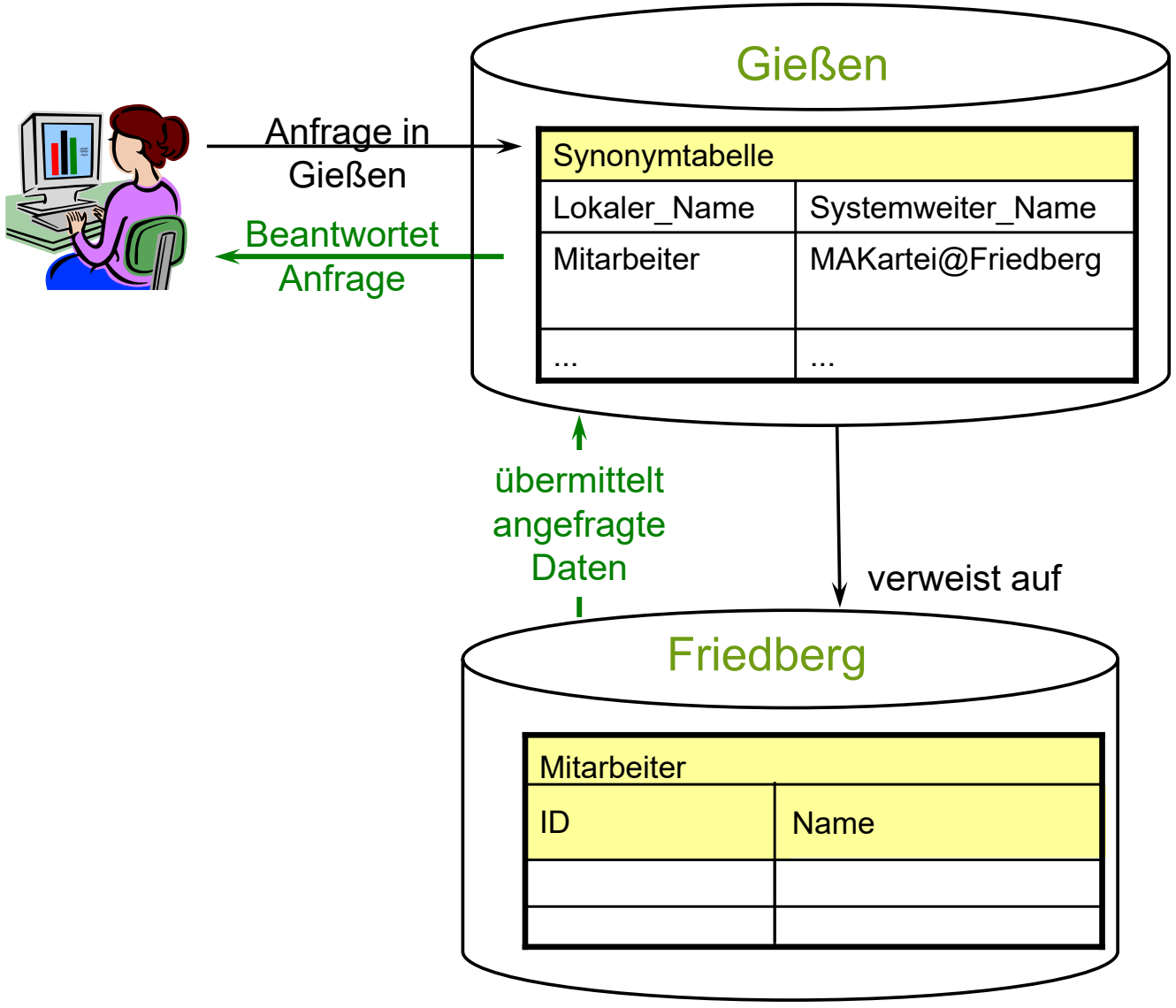
→ Systemweiter Name:



→ Synonym erstellen:

```
CREATE SYNONYM Mitarbeiter
FOR MAKartei@Friedberg
```


Verteilte DB
Datentabelle
Synonymtabelle
Lokaler_Name
Systemweiter_Name
...
...
Lokaler Katalog
Lokaler_Name
Systemweiter_Name
...
...


Gießen
Anfrage in Gießen
Beantwortet Anfrage
Synonymtabelle
Lokaler_Name
Systemweiter_Name
Mitarbeiter
MAKartei@Friedberg
...
...
übermittelt angefragte Daten
verweist auf
Friedberg
Mitarbeiter
ID
Name

Transaktionsmanagement

Ziel: Sicherstellung der Datenintegrität

Lösungsmöglichkeiten für Sperroperationen:

- Zentraler Knoten
  - Unabhängigkeit lokaler Komponenten nicht mehr gegeben
- Schreiben sperrt alles – Lesen sperrt eins
  Write locks all – Read locks one
  - Lokale Leseoperationen möglich, falls Relation vorhanden
  - Keine Schreibsperren bei ausgefallenem Knoten möglich
- Majoritätssperren
  - Lese-/Schreibsperre gilt als gesetzt, wenn Mehrheit der Relationen gesperrt werden kann
- Verwendung von Primär- und Sekundärkopien
  - Schreiben in Primärkopie, Lesen aus Sekundärkopie

Lösungsmöglichkeiten für Deadlockbehandlung:

- **Vermeidung** durch Reihenfolgespezifikation der Datenobjekte, z.B. sperren nur in alphabetischer Reihenfolge

  - T1 und T2 möchten gleichzeitig **A_Tabelle** und **B_Tabelle** sperren
  - T1 sperrt zunächst **A_Tabelle**
  - Es würde zu einem Deadlock kommen, wenn T2 nun zuerst **B_Tabelle** sperrt
  - Dies ist jedoch aufgrund der geforderten (alphabetischen) Reihenfolge nicht gestattet!

Lösungsmöglichkeiten für Deadlockbehandlung:

- **Entdeckung und Behebung**:
  - **Timeout**
  - Analyse von **Abhängigkeitsgraphen**
    - Standorte tauschen Informationen bzgl. wartender Transaktionen aus
    - Zyklen weisen auf Deadlocks hin

Teuer! - daher so gut wie nicht praktiziert

11 Transaktionsmanagement: Commit

- ACID setzt voraus, dass Transaktionen atomar sind
- Also entweder:

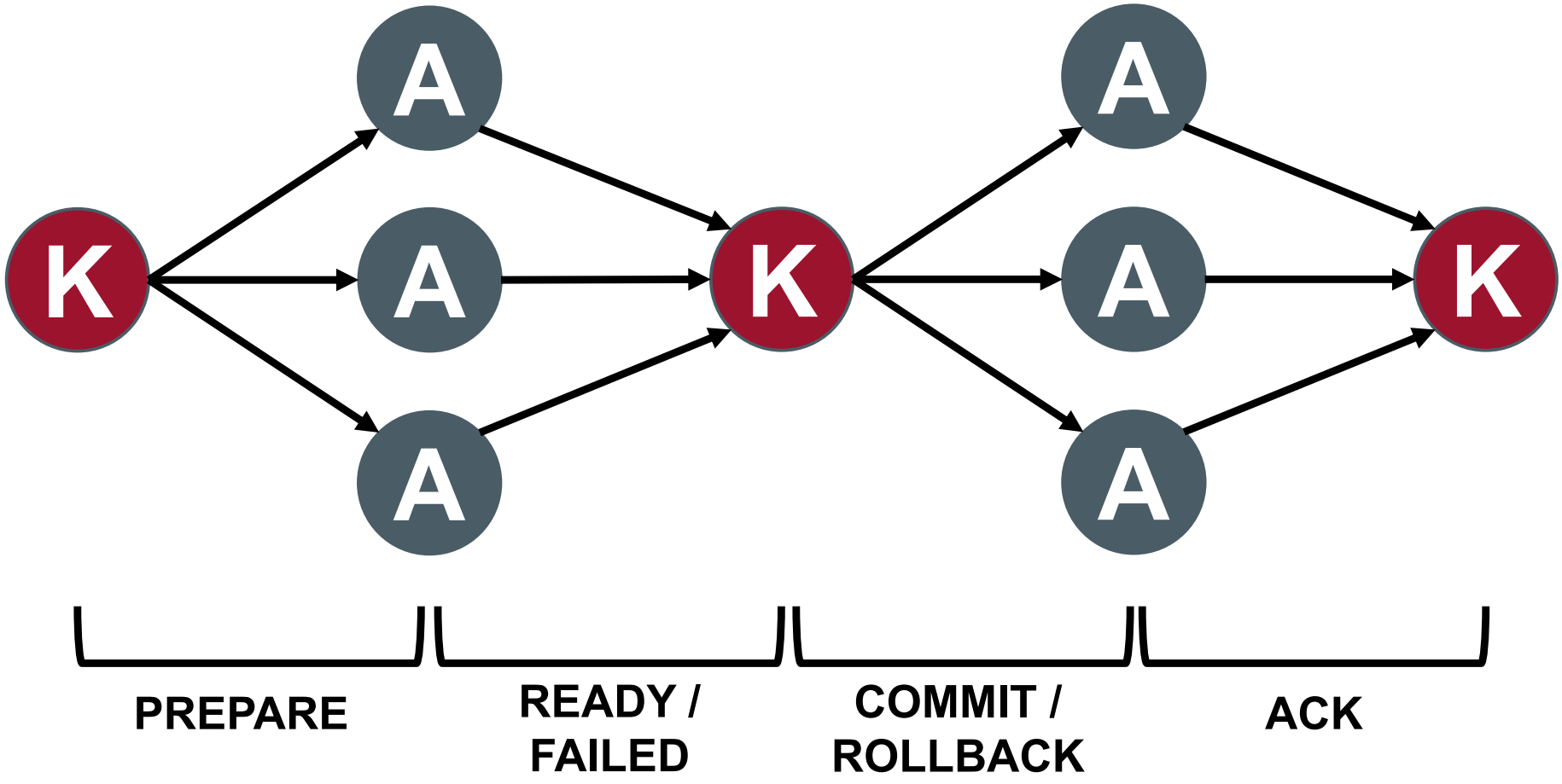
COMMIT ➔ Die verteilte Transaktion wird von allen beteiligten Knoten abgeschlossen.

- Oder:

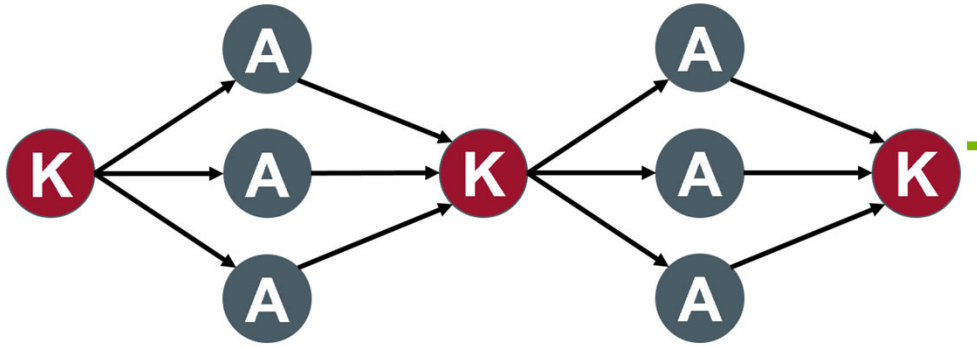
ROLLBACK ➔ Die Transaktion wird von allen Knoten abgebrochen.

Problem: in einem verteilten System
können alle Knoten unabhängig voneinander abstürzen.

11 2-Phase-Commit (2PC)



11 2-Phase-Commit (2PC)



Vorbereitung:

- Transaktionskoordinator sendet PREPARE TO COMMIT an alle beteiligten Stationen
- Empfänger: Logfile schreiben
- Rückmeldung: READY oder FAILED (oder Timeout)
- Koordinator: Fortsetzung oder Abbruch

Finales COMMIT:

- COMMIT an alle Stationen
- Empfänger: Änderung in Datenbank
- Rückmeldung: ACK.

Was wenn hier ein Teilnehmer ausfällt?

11 Das CAP-Theorem

- CAP ist ein Akronym für drei Eigenschaften:
  - Consistency
  - Availability
  - Partition Tolerance

Einheitlichkeit (Consistency) der Daten aller Knoten

- Immer konsistent (ACID)
- Schlussendlich konsistent (eventual consistency)

Verfügbarkeit (Availability) des Gesamtsystems

- Anfragen werden immer beantwortet
- Anfragen werden manchmal nicht beantwortet

Partitionstoleranz (Partition tolerance) der Knoten im Gesamtsystem

- Das System kann mit Partitionierung umgehen
- Das System versagt bei Partitionierung ganz oder teilweise

11 Das CAP-Theorem

- Kernaussage: In einem verteilten System ist es nicht möglich die drei Eigenschaften des CAP-Theorems gleichzeitig zu garantieren.
- Die Größen sind graduell, es gibt also kein „Ja oder Nein“.
- Im Jahr 2000 von Eric Brewer vermutet, 2002 axiomatisch bewiesen.
- Gilt nur im Fall von verteilten Systemen!

Weitere Infos: Understanding the CAP Theorem, Akhil Mehra, 2019
https://dzone.com/articles/understanding-the-cap-theorem

Der Umgang mit einer Netzwerkpartitionierung einer DB erfordert Abstriche in Einheitlichkeit oder Verfügbarkeit!

Bei ziemlich stabiler Hardware funktioniert es in der Praxis aber sehr gut

Big Data: Analysen und Auswertung



11 Hadoop und MapReduce

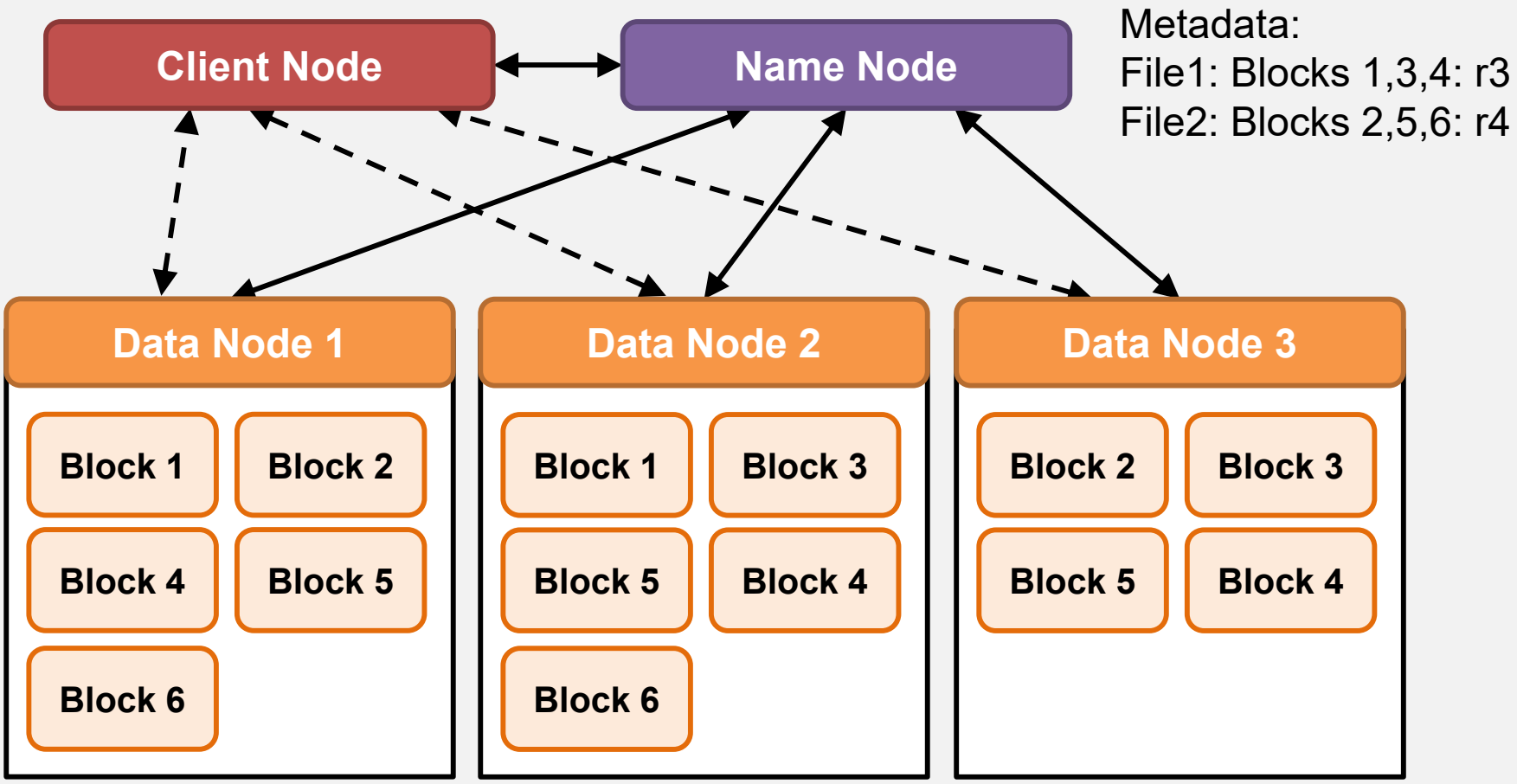
- **Hadoop**: De Facto Standard für eine skalierbare, verteilte Datenbank für Big Data
- Java-basiertes Framework für rechnerübergreifendes Speichern und Lesen von großen Datenmengen
- Wichtigste Komponenten des **Hadoop Frameworks**:
  - **Hadoop Distributed File System (HDFS)**
  - **MapReduce**

11 Hadoop Distributed File System (HDFS)

- Verteiltes Dateisystem für die hochperformante Verwaltung großer Datenmengen
- Optimiert für „***write-once, read-many***“
- Batch Processing
- Kontinuierlicher Datenstream:
  ➔ **Echtzeitanwendungen**
- **Ausfallsicherheit**: Replizierung der Daten
  - Standardmäßig wird jeder Datenblock auf 3 Servern gespeichert

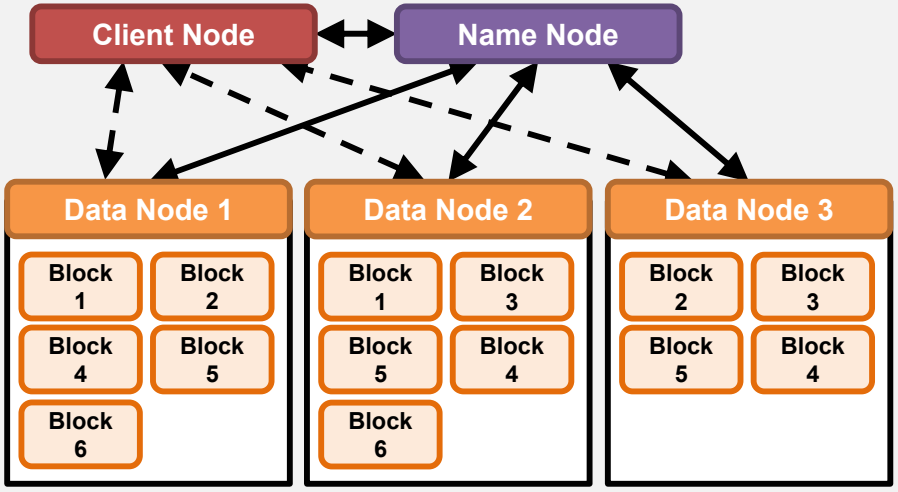
11 Hadoop Distributed File System (HDFS)

- Hadoop arbeitet auf mehreren Servern (Knoten/Nodes)
- Es werden **drei Arten von Nodes** unterschieden



Hadoop Distributed File System (HDFS)

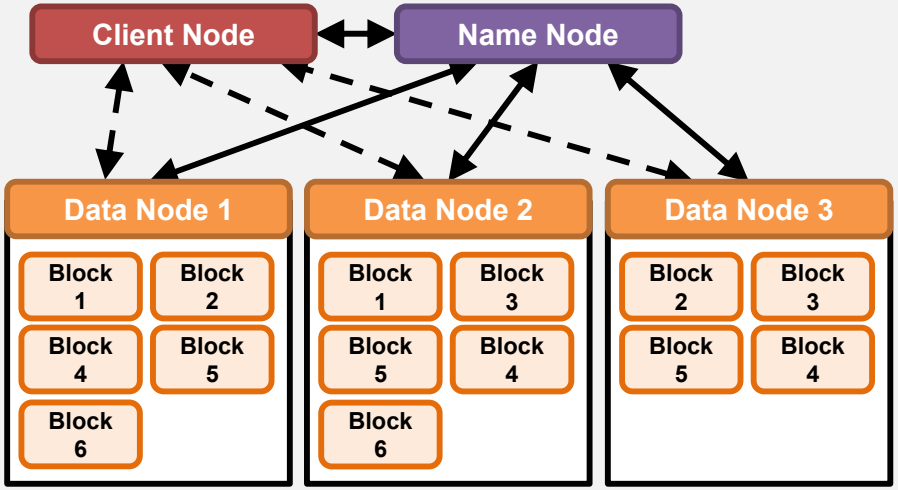
Name Node:



- Üblicherweise nur ein Name Node pro HDFS Cluster
- Enthält Metadaten für das Dateisystem
- Metadaten:
  - Name der zu speichernden Datei
  - Blöcke, in denen die Datei gespeichert wird
  - Replizierungsfaktor

Hadoop Distributed File System (HDFS)

Data Nodes:



- Speichern die eigentlichen Informationen im HDFS
- Dateien werden in Blocks gesplittet und auf Data Nodes verteilt
- Kommunikation mit Name Node:
  Heartbeat: Alle 3s ein Verfügbarkeitssignal („Lebenszeichen“)
  Block Report: Alle 6h Information über gespeicherte Blöcke

Client Node:

- **Speichern von Dateien**: Client Node --> Name Node (Anfrage)
- Aufgaben des Name Nodes
  - Speichern der Metadaten
  - Blocknummern werden für die Datei vergeben
  - Festlegung der Data Nodes, auf denen die Blöcke gespeichert werden sollen
  - Rückgabe dieser Informationen an Client Node
- Aufgaben des Client Nodes
  - Speichern der Informationen auf dem ersten Data Node, der durch den Name Node festgelegt wurde
  - Jeder Data Node wird darüber informiert, auf welchen anderen Data Nodes der Block repliziert wird
  - Data Nodes replizieren Informationen selbstständig

Client Node:

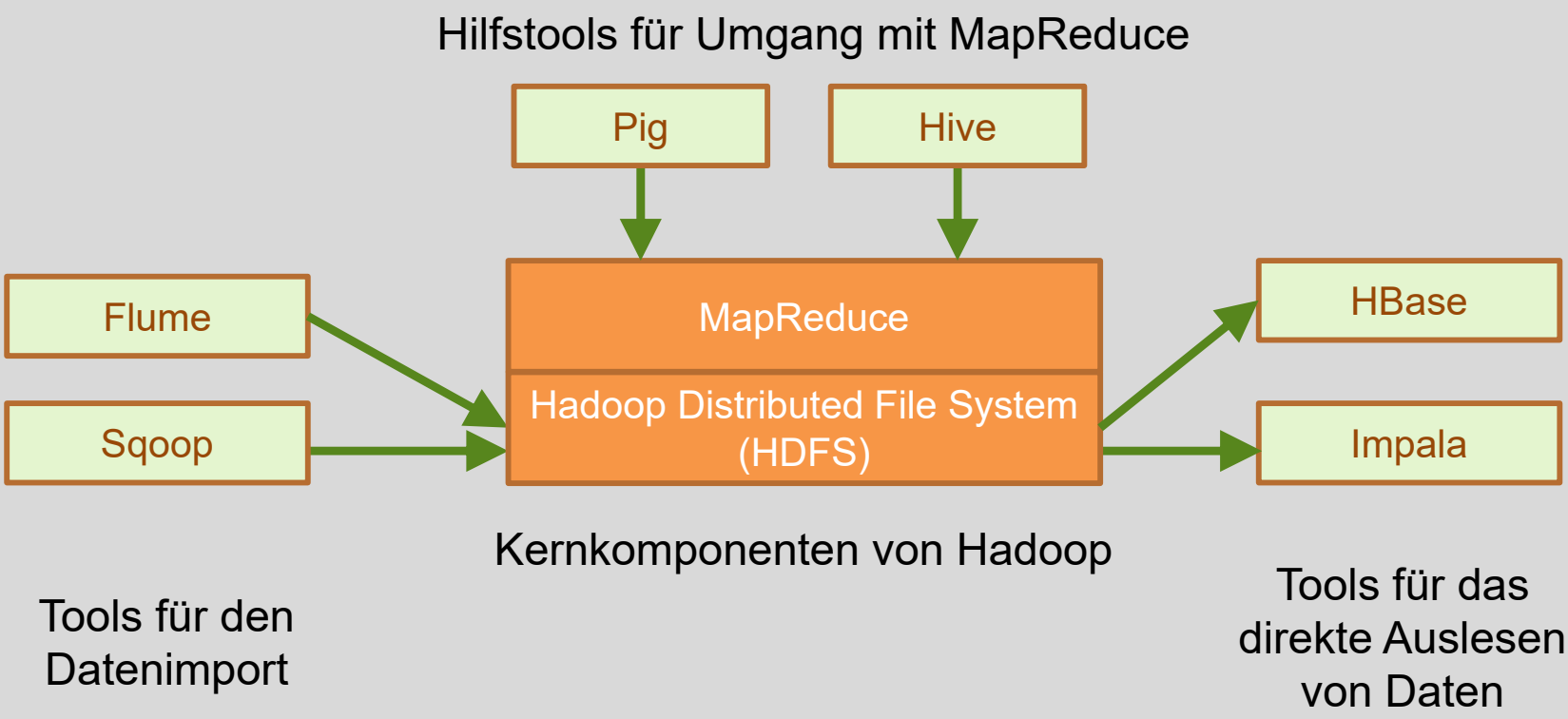
- **Lesen von Dateien**: Client Node --> Name Node
  Anfrage
- Aufgaben des Name Nodes
  - Ermittle alle Blöcke, die zu der Datei gehören
  - Ermittle den nächstgelegenen Data Node, der die entsprechenden Blöcke enthält
  - Gebe diese Informationen an Client Node zurück
- Aufgaben des Client Nodes
  - Direktes Lesen der Blöcke von Data Nodes

11 MapReduce

- **MapReduce** ist ein Open-Source-Framework, um große Datenmengen in einer verteilten Serverarchitektur wie HDFS zu verarbeiten
- Entwickelt durch Google
  - Beschrieben in einem Artikel aus 2008
- Name setzt sich zusammen aus zwei Funktionsbezeichnungen
  - **Map**: Sortierung und Filterung einer Datenmenge (z.B. heute erstellte Warenkorbpositionen)
  - **Reduce**: Aggregierung einer Datenmenge (z.B. Summenbildung)
- **MapReduce Jobs**: MapReduce-Datenaufbereitungen müssen in Java geschrieben werden

11 Hadoop Ökosystem

- Hadoop ist ein **Low-Level Tool**, dessen Installation, Wartung und Benutzung nicht enduserfreundlich ist
- Es wurden verschiedene Applikationen entwickelt, um den Umgang mit Hadoop zu erleichtern



Hilfstools für Umgang mit MapReduce

- Ohne Hilfstools sind Programmierkenntnisse für MapReduce Jobs notwendig
- Hilfstools sollen diese Hürde vereinfachen
- **Hive**
  - Ist ein Data Warehousing System für HDFS
  - HiveQL: SQL-ähnliche Syntax
  - Benutzer kann MapReduce Jobs mit Hilfe von HiveQL schreiben
- **Pig**
  - Pig Latin: Skriptsprache für MapReduce Jobs





Tools für den Datenimport

- Datenimport aus operativer Datenbank
- **Flume**
  - Hauptsächlich Server Logfiles
  - Automatisierter, zeitgesteuerter oder triggerbasierter Import
  - Unterstützt Transformation der Daten während des Imports



- **Sqoop**
  - Begriffszusammensetzung aus „SQL-to-Hadoop“
  - Ähnlich wie Flume, jedoch stammen die Quelldaten meist aus relationalen DB wie Oracle, MySQL oder SQL Server
  - Bidirektionaler Datenaustausch theoretisch möglich

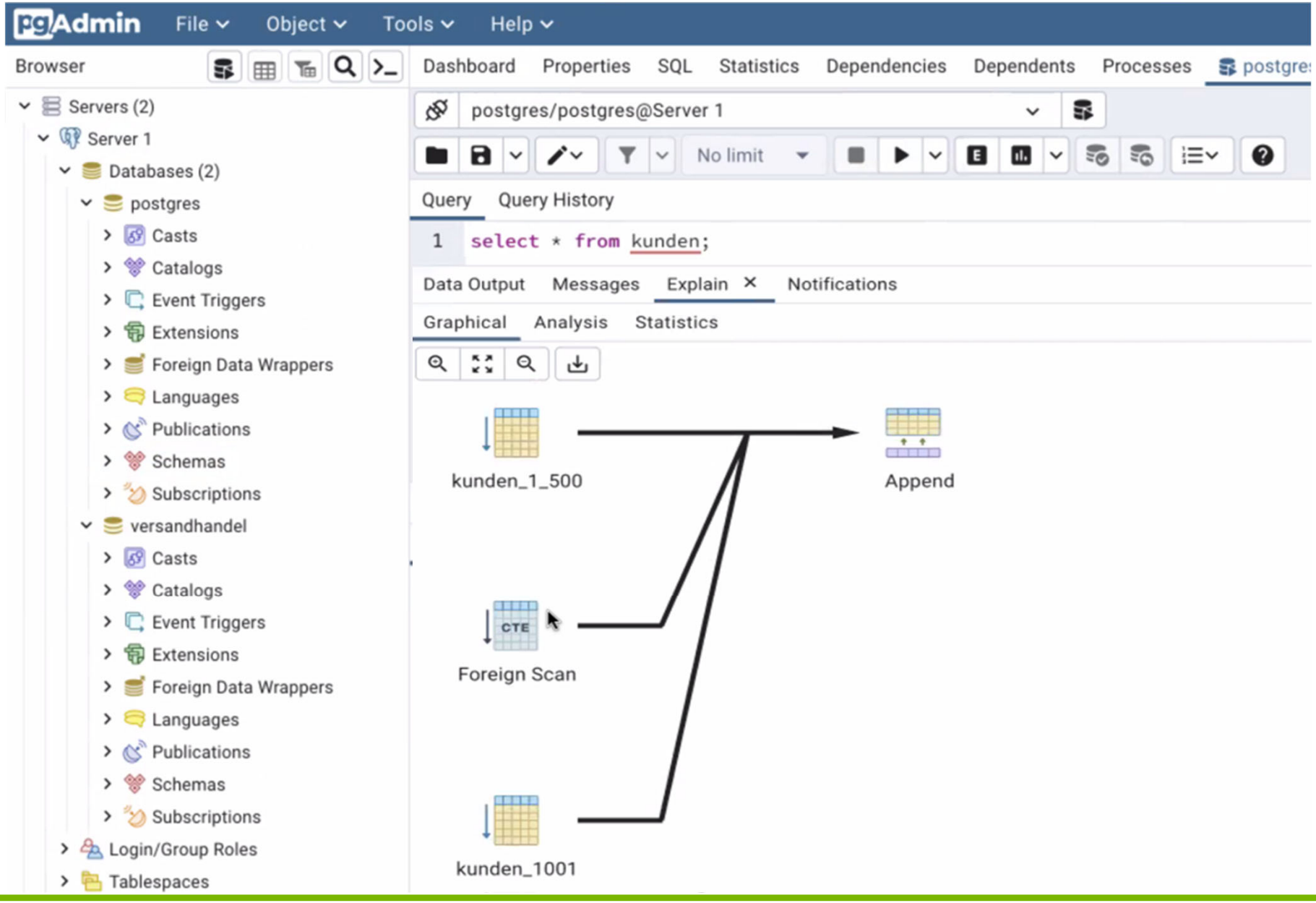


Tools für das direkte Auslesen von Daten

- Bedarf nach schnellerem Zugriff auf Daten im HDFS als MapReduce Jobs, die zumeist auf Daten-aggregation ausgelegt sind
- **HBase**
  - Spaltenorientierte NoSQL Datenbank, die auf HDFS ausgelegt ist
  - Optimiert für leichte Skalierung
  - Grundsätzlich werden Abfragen in Java programmiert werden, jedoch über Zusatzlayer auch SQL möglich
- **Impala**
  - War das erste Tool, das SQL Queries in Hadoop unterstütze



11 Verteilte DB – Eine Demo



11 Übung



1. Folgen Sie der Anleitung auf Moodle, um ein postgreSQL-Dump zu erstellen.

2. Setzen Sie ein neues Projekt mit Java oder Kotlin auf.
3. Finden Sie einen Weg, den aktuellen JDBC-Treiber für Postgres von *jdbc.postgresql.org* in Ihr Projekt einzubinden.
4. Legen Sie eine neue Datenbank im Playground des FBS an und importieren Sie aus den Vorlagen die Rezepte.
5. Fragen Sie alle Rezepte ab und geben Sie den Benutzernamen des Erstellers aus.
6. Geben Sie alle Benutzer aus, die ein Rezept mit 4 oder 5 Sternen haben.

11 Kontrollfragen

- Welche Datenbankzugriffstechniken kennen Sie?
- Was ist das ResultSet in JDBC? Wofür ist es gut?
- Auf welche Arten können wir Daten mit JDBC updaten?
- Wie können wir in JDBC Transaktionen nutzen?
- Was ist Big Data?
- Wann sollte man eine verteilte Datenbank nutzen?
- Wie funktioniert eine verteilte Datenbank?
- Was ist das Hadoop Ökosystem? Warum reicht nicht das verteilte Dateisystem HDFS?